ARCHITECTURE TECHNIQUE

Système de gestion de présence du personnel par QR Code

Frontend React · Backend Express.js · PostgreSQL avec node-postgres

Document de conception — version 1.0
Basé sur le PRD fourni

1. Décisions d'architecture

Le projet sera construit comme un monolithe modulaire dans un monorepo TypeScript. Cette solution sépare strictement l'interface et l'API tout en restant simple à développer, tester et déployer pour un MVP.

Couche	Technologie	Responsabilité
Frontend	React + TypeScript + Vite	Vues, formulaires, scan caméra, filtres et affichage.
État serveur	TanStack Query	Cache, chargement, synchronisation et invalidation des requêtes.
Routage	React Router	Routes publiques/protégées et contrôle d'accès visuel.
Backend	Node.js + Express + TypeScript	API REST, sécurité et orchestration métier.
Validation	Zod	Validation des entrées côté client et API.
Accès aux données	node-postgres pg + PostgreSQL	Requêtes SQL paramétrées, pool de connexions et transactions.
QR Code	html5-qrcode + qrcode	Lecture caméra côté frontend, génération côté backend.
PDF	PDFKit ou Puppeteer	Production serveur des fiches officielles.
Authentification	JWT access + refresh token en cookie HttpOnly	Sessions sécurisées des coordinateurs et secrétaires.
Planification	node-cron au MVP	Marquage quotidien des absences.

Règles transversales : heure du serveur comme référence, fuseau Indian/Antananarivo, QR opaque et révocable, mots de passe hachés, autorisations contrôlées dans l'API, journalisation des corrections manuelles.

2. Vue globale des échanges

```
Navigateur React
  ├─ /login, /dashboard, /agents, /scan, /presences, /reports...
  └─ HTTP JSON / téléchargement PDF
                ↓
API Express /api/v1
  Route → Middleware → Controller → Service → Repository → pg
                ↓
PostgreSQL

Scan : caméra → token QR → POST /scans → validation → transaction → résultat
```

3. Arborescence du monorepo

```
presence-qr/
├─ apps/
│  ├─ web/                  # Frontend React
│  │  └─ src/
│  │     ├─ app/             # router, providers, guards
│  │     ├─ layouts/           # AuthLayout, DashboardLayout
│  │     ├─ pages/             # vues routées
│  │     ├─ features/          # auth, agents, scans, présences...
│  │     ├─ components/          # composants partagés
│  │     ├─ services/          # client HTTP
│  │     ├─ hooks/             # hooks transversaux
│  │     ├─ schemas/             # schémas Zod
```

```
│   │   ├── types/              # types UI/API
│   │   └── utils/              # date, format, permissions
│   └── api/                    # Backend Express
│       └── src/
│           ├── config/         # env, logger, database
│           ├── modules/        # modules métier
│           ├── middlewares/    # auth, rôles, erreurs
│           ├── jobs/           # calcul automatique des absences
│           ├── shared/         # erreurs, pagination, types
│           └── app.ts / server.ts
├── packages/
│   └── contracts/              # enums, DTO et schémas partagés
├── database/
│   ├── migrations/             # scripts SQL versionnés
│   ├── seeds/                  # données initiales
│   └── schema.sql              # structure de référence
└── tests/                      # tests E2E et scénarios métier
```

4. Architecture frontend

4.1 Vues et routes

Route	Vue	Accès	Fonction
/login	LoginPage	Public	Connexion par email/identifiant et mot de passe.
/	DashboardPage	Coordinateur, Secrétaire	Indicateurs du jour, graphique hebdomadaire, derniers scans.
/scan	ScanPage	Coordinateur, Secrétaire	Caméra, saisie de secours, confirmation ou erreur du scan.
/agents	AgentListPage	Coordinateur	Liste, recherche, filtre par service/statut, pagination.
/agents/new	AgentCreatePage	Coordinateur	Création d'un agent et affectation à un service.
/agents/:id	AgentDetailsPage	Coordinateur	Fiche agent, QR actif, historique récent.
/agents/:id/edit	AgentEditPage	Coordinateur	Modification et activation/désactivation.
/agents/:id/qr	AgentQrPage	Coordinateur	Afficher, générer, régénérer et imprimer le QR.
/presences	PresenceListPage	Coordinateur, Secrétaire	Présences par date avec filtres et corrections autorisées.
/presences/:id	PresenceDetailsPage	Coordinateur, Secrétaire	Détail d'une journée agent et historique des modifications.
/absences	AbsenceListPage	Coordinateur, Secrétaire	Agents absents, justification et correction.
/retards	LateListPage	Coordinateur, Secrétaire	Retards et durée calculée.
/history	HistoryPage	Coordinateur, Secrétaire	Recherche multi-critères sur une période.
/reports	ReportListPage	Coordinateur, Secrétaire	Fiches journalières archivées et export.
/reports/daily	DailyReportPage	Coordinateur, Secrétaire	Prévisualisation de la fiche d'une date.

Route	Vue	Accès	Fonction
/users	UserListPage	Coordinateur	Comptes internes et rôles.
/users/new	UserCreatePage	Coordinateur	Création d'un secrétaire ou coordinateur.
/settings	SettingsPage	Coordinateur	Horaires, tolérance, seuil d'absence et identité de l'organisation.
/profile	ProfilePage	Coordinateur, Secrétaire	Profil et changement de mot de passe.
/403	ForbiddenPage	Authentifié	Accès refusé.
*	NotFoundPage	Tous	Route introuvable.

4.2 Layouts, guards et composants partagés

Élément	Type	Fonctions / props essentielles
AuthLayout	Layout	Cadre des écrans de connexion.
DashboardLayout	Layout	Sidebar, topbar, zone de contenu, menu selon rôle.
ProtectedRoute	Guard	Vérifie isAuthenticated; redirige vers /login.
RoleGuard	Guard	Vérifie hasRole(allowedRoles); redirige vers /403.
AppErrorBoundary	Classe React	componentDidCatch(), affichage d'un fallback.
DataTable	Composant	columns, rows, sort, pagination, loading, emptyState.
FilterBar	Composant	date, période, agent, service, statut; onChange(), onReset().
StatusBadge	Composant	Affiche PRESENT, LATE, ABSENT, JUSTIFIED.
StatCard	Composant	label, value, trend, icon.
ConfirmDialog	Composant	open, title, message, onConfirm(), onCancel().
ToastProvider	Provider	success(), error(), warning(), info().
QrScanner	Composant	start(), stop(), onDecode(token), onError(error).
QrPreview	Composant	qrImageUrl, agentCode, print().
PdfViewer	Composant	url, download(), print().
LoadingState / EmptyState	Composant	États cohérents des listes.
FormField	Composant	label, error, required, children.

4.3 Services frontend

Classe / module	Fonctions nécessaires
ApiClient	get(url, params), post(url, body), patch(url, body), delete(url), download(url), refreshToken(), setUnauthorizedHandler().
AuthService	login(credentials), logout(), refresh(), getCurrentUser(), changePassword(payload).
AgentService	list(filters), getById(id), create(data), update(id,data), activate(id), deactivate(id), getHistory(id).
QrCodeService	getActive(agentId), generate(agentId), regenerate(agentId), download(agentId), print(agentId).
ScanService	submitToken(token, clientScanId), getRecent(limit).
PresenceService	list(filters), getById(id), update(id,data), justifyAbsence(id,data), correct(id,data).

Classe / module	Fonctions nécessaires
DashboardService	getSummary(date), getWeeklyStats(range), getRecentScans().
ReportService	previewDaily(date,filters), downloadDailyPdf(date), listArchived(filters), exportCsv(filters).
UserService	list(), create(data), update(id,data), resetPassword(id), activate(id), deactivate(id).
SettingsService	get(), update(data), getServices(), createService(data), updateService(id,data).

4.4 Hooks et fonctions utilitaires

Hook / module	Fonctions
useAuth	user, isAuthenticated, login(), logout(), can(permission).
useAgents	useAgentList(filters), useAgent(id), useCreateAgent(), useUpdateAgent().
useQrScanner	startScanner(), stopScanner(), isScanning, permissionState, lastToken.
useScanMutation	scan(token), isPending, result, reset(); verrou anti double-envoi.
usePresences	usePresenceList(filters), usePresence(id), useCorrectPresence().
useReports	useDailyReport(date), downloadPdf(), exportCsv().
useDebounce	useDebounce(value, delay) pour la recherche.
usePagination	page, pageSize, setPage(), resetPage().
permissions.ts	hasRole(user,roles), can(user,permission).
date.ts	formatDate(), formatTime(), toApiDate(), getLocalToday().
errors.ts	getErrorMessage(error), mapValidationErrors(error).
download.ts	saveBlob(blob, filename), openPrintWindow(url).

5. Architecture backend

Chaque module métier possède route, controller, service, repository, DTO/schema et tests. Le controller traite HTTP; le service porte les règles; le repository exécute des requêtes SQL paramétrées avec le module pg.

Module	Controller / méthodes	Service / fonctions métier	Repositories
Auth	AuthController: login(), refresh(), logout(), me(), changePassword()	AuthService: authenticate(), issueTokens(), rotateRefreshToken(), revokeSession(), verifyPassword(), changePassword()	UserRepository, SessionRepository
Agents	AgentController: list(), getOne(), create(), update(), activate(), deactivate(), history()	AgentService: listAgents(), getAgent(), createAgent(), updateAgent(), setActiveState(), assertUniqueCode()	AgentRepository, AuditRepository
QR Codes	QrCodeController: getActive(), generate(), regenerate(), download()	QrCodeService: getActiveToken(), generateForAgent(), regenerateForAgent(), validateToken(), revokeTokens(), renderQrPng()	QrTokenRepository, AgentRepository

Module	Controller / méthodes	Service / fonctions métier	Repositories
Scans	ScanController: scan(), recent()	ScanService: processScan(), resolveAgent(), determineScanType(), assertScanAllowed(), createArrival(), createDeparture()	QrTokenRepository, PresenceRepository, ScanEventRepository
Présences	PresenceController: list(), getOne(), update(), correct(), justifyAbsence()	PresenceService: listPresences(), getPresence(), correctPresence(), justifyAbsence(), recalculateStatus(), validateChronology()	PresenceRepository, AuditRepository
Absences	AbsenceController: list(), computeForDate()	AbsenceService: markDailyAbsences(), findExpectedAgents(), createMissingAbsences(), recomputeDate()	AgentRepository, PresenceRepository
Dashboard	DashboardController: summary(), weekly(), recentScans()	DashboardService: getDailySummary(), getWeeklyTrend(), getRecentActivity()	PresenceRepository, AgentRepository
Rapports	ReportController: dailyPreview(), dailyPdf(), exportCsv(), archives()	ReportService: buildDailyReport(), generateDailyPdf(), generateCsv(), archiveReport()	PresenceRepository, SettingsRepository, ReportRepository
Utilisateurs	UserController: list(), create(), update(), activate(), deactivate(), resetPassword()	UserService: createUser(), updateUser(), setActiveState(), resetPassword(), assertRoleAssignable()	UserRepository, AuditRepository
Paramètres	SettingsController: get(), update(), listServices(), createService(), updateService()	SettingsService: getSettings(), updateSettings(), validateSchedule(), createDepartment(), updateDepartment()	SettingsRepository, DepartmentRepository
Audit	AuditController: list()	AuditService: record(), listEvents()	AuditRepository

5.1 Classes transversales et middlewares

Élément	Fonctions / responsabilité
AppError extends Error	code, statusCode, details; erreurs métier standardisées.
ErrorHandlerMiddleware	errorHandler(err,req,res,next), masque les détails internes.
AuthMiddleware	authenticate(req,res,next), charge req.user depuis le JWT.
RoleMiddleware	authorize(...roles), refuse les opérations non autorisées.
ValidationMiddleware	validate(schema, source), transforme et valide body/query/params.
RateLimitMiddleware	Limite login et endpoint de scan.
RequestIdMiddleware	Ajoute un identifiant de corrélation aux logs.
AuditMiddleware / AuditService	Enregistre auteur, action, cible, ancienne/nouvelle valeur.
DatabasePool	Pool pg partagé, query(text,params), getClient() et fermeture propre.

Élément	Fonctions / responsabilité
TransactionManager	withTransaction(callback), BEGIN, COMMIT, ROLLBACK et libération du client.
PasswordHasher	hash(password), compare(password,hash).
TokenProvider	signAccessToken(), verifyAccessToken(), createRefreshToken().
Clock	now(), today(); facilite les tests des horaires.
PdfGenerator	generateDailyAttendance(data,organization), retourne Buffer/stream.
Pagination	parsePagination(query), buildPageResult(items,total,page,size).

5.2 Contrats des principales fonctions métier

Fonction	Entrée	Sortie / règle
ScanService.processScan	qrToken, clientScanId, actorId	ScanResult. Transaction atomique et idempotente.
determineScanType	présence du jour	Aucune arrivée → ARRIVAL; arrivée sans départ → DEPARTURE; sinon rejet.
calculateArrivalStatus	arrivalTime, workStart, tolerance	PRESENT si arrivée ≤ début + tolérance, sinon LATE.
calculateLateMinutes	arrivalTime, workStart, tolerance	Nombre de minutes après la limite; minimum 0.
validateChronology	arrivalAt, departureAt	Départ strictement postérieur à l'arrivée.
markDailyAbsences	date, absenceCutoff	Crée ABSENT pour chaque agent actif attendu sans présence.
correctPresence	presenceId, patch, reason, actorId	Correction + recalcul + entrée d'audit obligatoire.
generateForAgent	agentId	Révoque l'ancien QR si nécessaire; crée un token aléatoire haché.
buildDailyReport	date, filters	Lignes ordonnées + totaux présents/retards/absents.
generateDailyPdf	date, filters	PDF avec identité de l'organisation et tableau.

6. Modèle de données

Entité	Champs essentiels	Relations / contraintes
User	id, name, email, passwordHash, role, isActive, createdAt	email unique; role COORDINATOR ou SECRETARY.
Department	id, name, code, isActive	name/code uniques; possède plusieurs agents.
Agent	id, employeeCode, firstName, lastName, departmentId, position, isActive, createdAt	employeeCode unique; appartient à Department.
QrToken	id, agentId, tokenHash, version, isActive, issuedAt, revokedAt	Un seul token actif par agent; cascade contrôlée.
Attendance	id, agentId, date, arrivalAt, departureAt, status, lateMinutes, source, note	Unique(agentId,date); dates stockées en UTC.
ScanEvent	id, attendanceId, agentId, type, scannedAt, clientScanId, result	clientScanId unique pour idempotence.

Entité	Champs essentiels	Relations / contraintes
WorkSchedule	id, dayOfWeek, startTime, endTime, toleranceMinutes, absenceCutoff, isWorkingDay	Configuration par jour de semaine.
AppSetting	id, key, valueJson, updatedBy, updatedAt	Paramètres organisationnels.
ReportArchive	id, date, fileName, checksum, generatedBy, generatedAt	Optionnel au MVP si génération à la demande.
RefreshSession	id, userId, tokenHash, expiresAt, revokedAt	Rotation et révocation des sessions.
AuditLog	id, actorId, action, entityType, entityId, beforeJson, afterJson, reason, createdAt	Immuable; corrections et actions sensibles.

6.1 Énumérations

```
UserRole       = COORDINATOR | SECRETARY
AttendanceStatus = PRESENT | LATE | ABSENT | JUSTIFIED
ScanType       = ARRIVAL | DEPARTURE
AttendanceSource = QR_SCAN | MANUAL | SYSTEM
ScanResult     = ACCEPTED | REJECTED | DUPLICATE | INVALID_TOKEN
AuditAction    = CREATE | UPDATE | ACTIVATE | DEACTIVATE | REGENERATE_QR | CORRECT
```

7. Endpoints REST

Méthode	Endpoint /api/v1	Accès	Action
POST	/auth/login	Public	Connexion
POST	/auth/refresh	Cookie	Renouveler le jeton
POST	/auth/logout	Authentifié	Fermer la session
GET	/auth/me	Authentifié	Utilisateur courant
GET	/agents	Coordinateur	Lister/filtrer
POST	/agents	Coordinateur	Créer
GET	/agents/:id	Coordinateur	Consulter
PATCH	/agents/:id	Coordinateur	Modifier
PATCH	/agents/:id/status	Coordinateur	Activer/désactiver
GET	/agents/:id/qr	Coordinateur	QR actif
POST	/agents/:id/qr	Coordinateur	Générer
POST	/agents/:id/qr/regenerate	Coordinateur	Régénérer
GET	/agents/:id/qr/download	Coordinateur	Télécharger
POST	/scans	Coordinateur, Secrétaire	Traiter un scan
GET	/scans/recent	Coordinateur, Secrétaire	Scans récents
GET	/presences	Coordinateur, Secrétaire	Lister/filtrer
GET	/presences/:id	Coordinateur, Secrétaire	Détail
PATCH	/presences/:id/correct	Coordinateur	Corriger avec motif
PATCH	/presences/:id/justify	Coordinateur, Secrétaire	Justifier une absence
GET	/absences	Coordinateur, Secrétaire	Absences
POST	/absences/recompute	Coordinateur	Recalcul manuel
GET	/dashboard/summary	Coordinateur, Secrétaire	Indicateurs
GET	/dashboard/weekly	Coordinateur, Secrétaire	Tendance
GET	/reports/daily	Coordinateur, Secrétaire	Prévisualiser
GET	/reports/daily.pdf	Coordinateur, Secrétaire	Télécharger PDF
GET	/reports/export.csv	Coordinateur, Secrétaire	Exporter CSV
GET	/users	Coordinateur	Lister comptes
POST	/users	Coordinateur	Créer compte
PATCH	/users/:id	Coordinateur	Modifier compte

Méthode	Endpoint /api/v1	Accès	Action
GET	/settings	Coordinateur	Lire paramètres
PATCH	/settings	Coordinateur	Modifier paramètres
GET	/departments	Authentifié	Lister services
POST	/departments	Coordinateur	Créer service
GET	/audit-logs	Coordinateur	Consulter le journal

8. Workflow du scan

1. Le composant QrScanner lit le token et suspend temporairement la caméra.

2. Le frontend crée un clientScanId UUID et appelle POST /api/v1/scans.

3. L'API authentifie l'opérateur, valide le format et applique le rate limiting.

4. QrCodeService compare le hachage du token à un QR actif et retrouve l'agent.

5. ScanService ouvre une transaction et verrouille la présence du jour.

6. Sans arrivée : création ARRIVAL et calcul PRESENT/LATE. Avec arrivée sans départ : mise à jour DEPARTURE. Avec arrivée et départ : rejet.

7. ScanEvent est enregistré; la réponse contient le nom, le type, l'heure et le statut.

8. L'interface affiche un toast vert ou un message d'erreur, puis réactive la caméra après un court délai.

Réponse recommandée : { success, eventType, scannedAt, attendanceStatus, agent: { id, employeeCode, displayName }, message }.

9. Tâches planifiées et règles temporelles

- Job mark-absences : exécuté après absenceCutoff les jours ouvrés; relançable sans doublon.
- Les jours non ouvrés ne créent aucune absence.
- Les heures sont comparées avec le fuseau Indian/Antananarivo, mais stockées en UTC.
- Un changement d'horaire ne modifie pas automatiquement les anciennes fiches; une correction explicite est auditée.
- Une présence ne peut contenir qu'une arrivée et un départ. Les scans supplémentaires sont rejetés.
- Un QR régénéré invalide immédiatement le précédent.

10. Sécurité et intégrité

- Token QR aléatoire d'au moins 128 bits; seule son empreinte est stockée.
- Mot de passe haché avec Argon2 ou bcrypt; politique minimale côté API.
- Refresh token dans un cookie HttpOnly, Secure et SameSite; access token court.
- Validation Zod sur body, params et query; aucune confiance dans le frontend.
- RBAC sur chaque endpoint sensible; le menu masqué ne remplace jamais l'autorisation serveur.
- Transactions PostgreSQL via pg pour le scan et contraintes uniques pour bloquer les doublons.
- Rate limiting sur connexion et scan, CORS limité à l'origine du frontend, Helmet activé.

- Audit obligatoire pour correction, justification, changement d'horaire et régénération de QR.
- Sauvegardes PostgreSQL et restauration testée périodiquement.

11. Tests nécessaires

Niveau	Cas prioritaires
Unitaires	calculateArrivalStatus, calculateLateMinutes, determineScanType, validateChronology, permissions.
Services	Premier scan, second scan, troisième scan rejeté, QR révoqué, agent inactif, absence, correction auditée.
API / intégration	Authentification, rôles, validation, filtres, pagination, PDF, transaction concurrente.
Frontend	Formulaires, guards, filtres, toast de scan, erreurs caméra, états loading/empty/error.
E2E	Connexion → création agent → QR → arrivée → départ → fiche → PDF.
Sécurité	Accès secrétaire aux routes coordinateur, token expiré, brute force, injection et CORS.

12. Ordre d'implémentation recommandé

Lot	Contenu
1 — Socle	Monorepo, module pg, PostgreSQL, scripts SQL, configuration, erreurs, validation, authentification et rôles.
2 — Référentiel	Services/départements, agents, utilisateurs et paramètres horaires.
3 — QR et scan	Génération/révocation QR, caméra, processus arrivée/départ et confirmations.
4 — Présences	Liste, filtres, retards, absences, corrections et audit.
5 — Pilotage	Dashboard, historique, statistiques et scans récents.
6 — Documents	Fiche journalière, PDF, impression et export CSV.
7 — Robustesse	Tests E2E, sécurité, logs, sauvegarde, CI et déploiement.

13. Critères de fin du MVP

- Un coordinateur peut gérer agents, comptes, services, horaires et QR Codes.
- Un coordinateur ou secrétaire peut scanner un QR et obtenir une confirmation immédiate.
- Le système distingue arrivée/départ, présent/retard/absent sans incohérence.
- Les corrections sont justifiées et historisées.
- Les listes sont filtrables par date, période, agent, service et statut.
- La fiche journalière est visualisable, imprimable et exportable en PDF.
- Les permissions sont vérifiées côté API et les scénarios critiques disposent de tests.